■# ChronosMatch - Final Project Report

architecture.md

ChronosMatch Architecture

Overview

ChronosMatch is a zero-copy high-frequency trading engine. It uses a shared-memory ring buffer to transfer orders between the simulator and matching engine.

System Architecture

flowchart TD

A[Order Simulator] --> B[Shared Memory Ring Buffer]

B --> C[Matching Engine]

C --> D[Order Matching]

D --> E[Trade Execution]

E --> F[Trade Database]

C --> G[Statistics]

G --> H[Market Dashboard]

F --> H

Components

1. Order Simulator

Generates buy and sell orders for testing the matching engine.

2. Shared Memory Ring Buffer

Transfers orders between the simulator and matching engine. It avoids unnecessary data copying.

3. Matching Engine

Receives orders and processes them according to the matching logic.

4. Order Matching

Matches compatible buy and sell orders based on price and order rules.

5. Trade Execution

Creates trade records when orders are successfully matched.

6. Trade Database

Stores trade information for later review and analysis.

7. Statistics

Collects engine performance metrics such as orders processed and orders per second.

8. Market Dashboard

Displays order book information, activity, engine status, and performance metrics.

Data Flow

1. The simulator generates orders.
2. Orders are placed into the shared memory ring buffer.
3. The matching engine reads and processes the orders.
4. Compatible buy and sell orders are matched.
5. Executed trades are recorded in the database.
6. Statistics are collected and displayed on the dashboard.

Summary

ChronosMatch uses a modular architecture in which the simulator, ring buffer, matching engine, trade database, statistics, and dashboard work together to process and display market orders.

data_contract.md

ChronosMatch Data Contract v1.1

1. Purpose

This document defines the data contract between ChronosMatch components. It specifies record formats, shared-memory communication, file responsibilities, and buffer handling rules.

2. Record Types

ChronosMatch uses two record types:

• **Order Record:** Fixed size of 32 bytes.
• **Trade Record:** Fixed size of 64 bytes.

Each record must follow its defined binary format to ensure consistent communication between components.

3. Order Record

• **Record size:** 32 bytes
• **Format:** `<QQIB3xQ`
• **Python module:** `ipc/record.py`

The order record contains order identifiers, price, quantity, side, and additional order data as defined by the existing record implementation.

4. Trade Record

• **Record size:** 64 bytes

• **Format:** `<QQIB3xQQQQ8x`

• **Python module:** `ipc/trade.py`

Trade records store trade-related information using the fixed 64-byte binary format. Packing and unpacking must follow the format defined in `ipc/trade.py`.

5. Ring Buffer Contract

• **Implementation:** `ipc/ring_buffer.py`

• The ring buffer supports configurable `record_size`.

• The default record size is 32 bytes for order records.

• A record size of 64 bytes can be used for trade records.

• Records must have the expected size before being written.

• The buffer must preserve FIFO (First In, First Out) ordering.

• A full buffer must reject new records rather than overwrite unread records.

• An empty buffer must not return a record.

6. File Responsibilities

| File | Responsibility |

|---|---|

| `ipc/record.py` | Order record format and pack/unpack functions |

| `ipc/trade.py` | Trade record format and pack/unpack functions |

| `ipc/ring_buffer.py` | Shared-memory ring buffer implementation |

| `docs/record_spec.md` | Record layout and field specifications |

| `docs/data_contract.md` | Data contract and communication rules |

| `tests/test_ring_buffer.py` | Ring buffer behavior tests |

7. Create vs. Attach Policy

• The component responsible for initializing the shared-memory region creates the buffer.

• Other components that need access to the same region attach to the existing shared-memory buffer.

• Components must use the same buffer layout, capacity, and record size.

• A component must not independently initialize or reset a buffer that is already in use by another component.

8. Full-Buffer Policy

When the ring buffer is full, a new record must not overwrite an unread record. The push operation must indicate failure so that the calling component can handle the situation.

9. Data Consistency

• All components must use the agreed record formats.

• Order records must remain fixed at 32 bytes.

• Trade records must remain fixed at 64 bytes.
• Any record format change must be reflected in the implementation and documentation.

10. Stats Block Contract

• **Implementation:** `ipc/stats.py`
• **File:** `stats.bin`
• **Purpose:** Stores shared trading statistics for readers such as monitoring or dashboard components.
• **Total size:** 576 bytes (64 latency buckets x 8 bytes each).
• **Magic:** `CHST`
• **Version:** `1`

Header Layout

The header uses the little-endian format `<4sIQQQQQQ`.

Field	Type	Size
Magic	4-byte value	4 bytes
Version	Unsigned integer	4 bytes
Trades	Unsigned 64-bit integer	8 bytes
Orders	Unsigned 64-bit integer	8 bytes
Best bid	Unsigned 64-bit integer	8 bytes
Best ask	Unsigned 64-bit integer	8 bytes
Last trade price	Unsigned 64-bit integer	8 bytes
Last trade quantity	Unsigned 64-bit integer	8 bytes
Heartbeat timestamp	Unsigned 64-bit integer	8 bytes

Latency Buckets

• The stats block contains 64 latency buckets.
• Each bucket is an unsigned 64-bit integer (8 bytes).
• Each bucket stores a counter.
• Valid bucket indexes are `0` to `63`.
• An invalid bucket index must raise `ValueError`.

Create and Attach Rules

• The initializing component creates and initializes `stats.bin` using `create=True`.
• Readers attach to the existing file using `create=False`.
• The file must have the expected size, magic, and version.

• Readers must not independently reset an active stats file.

• The matching engine should be the only writer, as specified in `ipc/stats.py`.

11. Week 1â€“2 Freeze Verification

Verified

• Order records use the 32-byte format `<QQIB3xQ`.

• Trade records use the 64-byte format `<QQIB3xQQQQ8x`.

• The stats header is 64 bytes, followed by 64 latency buckets of 8 bytes each.

• Total stats block size is 576 bytes.

• The simulator successfully sent and received 200,000 orders in the verification run.

• The full Python test suite passed: 39 tests.

Pending

• End-to-end integration between the simulator, matching engine, and dashboard has not yet been verified.

• The dashboard directory currently contains only a README file.

• The engine directory currently contains `ledger.py` and `test_ledger.py`; a complete matching-engine entry point has not been identified.

The Week 1â€“2 freeze is not fully complete until the missing integration components and their interfaces are verified.

zero_copy.md

Zero-Copy Data Flow in ChronosMatch

1. Introduction

ChronosMatch uses a memory-mapped ring buffer to share fixed-size records between processes.

The ring buffer is implemented using Python's `mmap` module. It supports both normal record copying and zero-copy record access.

Zero-copy access reduces unnecessary data copying when reading records from the mapped buffer.

2. Memory-Mapped Ring Buffer

The ring buffer is stored in a file and mapped into memory using `mmap`.

Its layout contains:

• A 64-byte header.

• A fixed-size record area.

• Head and tail counters to track produced and consumed records.

• A capacity value to define the maximum number of records.

The producer writes records into the buffer, and the consumer reads and consumes them.

3. Zero-Copy Reading

ChronosMatch provides two methods for zero-copy reading.

peek_view()

The `peek_view()` method returns a Python `memoryview` that directly references the next record inside the memory-mapped file.

It does not create a separate copy of the record.

Example:

```
view = ring.peek_view()

if view is not None:
    print(view)
```

advance()

The `advance()` method moves the tail counter to consume the current record.

It does not copy the record data.

Example:

```
view = ring.peek_view()

if view is not None:
    try:
        process_record(view)
    finally:
        view.release()

    ring.advance()
```

The memoryview should be released before consuming or reusing the underlying buffer slot.

4. Normal Copying Method

The `pop()` method reads the next record and converts it into a new `bytes` object.

Example:

```
record = ring.pop()
```

This creates a copy of the record data.

The simulator currently uses this normal copying method when reading trade records from the trade buffer. The former `ipc/check_pipe.py` script has been removed; its pipe audit is now covered by `tests/test_pipe_10k.py`.

5. Data Flow in the Trade Journal

The `ipc/journal.py` module uses the zero-copy reading methods.

Its processing flow is:

1. Read the next trade using `peek_view()`.
2. Decode the memoryview using `unpack_trade()`.
3. Release the memoryview.
4. Insert the decoded trade into SQLite.
5. Commit the database transaction.
6. Consume the trade using `advance()`.

Although the ring-buffer read uses a zero-copy memoryview, decoding creates Python data objects and SQLite performs database storage operations. Therefore, the complete journaling pipeline is not zero-copy.

6. Where Copying Happens

Operation	Data handling
`mmap` access	Shared mapped memory
`peek_view()`	Zero-copy memoryview
`advance()`	Updates tail counter without copying
`pop()`	Copies record into a bytes object
`push()`	Copies supplied record bytes into mapped memory
`unpack_trade()`	Decodes record into Python data
SQLite insertion	Stores decoded values in the database
Order ledger	JSON serialization and file writing

7. Benchmarking

The `ipc/bench_ring.py` module compares:

• Normal copying using `pop()`.
• Zero-copy reading using `peek_view()` and `advance()`.

The benchmark helps evaluate the performance difference between the two approaches.

Actual performance depends on the system, workload, and record size.

8. Conclusion

ChronosMatch supports zero-copy reading through memory-mapped files, `peek_view()`, and `advance()`.

The approach reduces record copying during ring-buffer consumption. However, decoding, database insertion, and other processing steps may still involve data conversion or copying.

Therefore, ChronosMatch uses zero-copy access in selected parts of its data flow rather than providing a completely zero-copy pipeline.

record_spec.md

Order Record Specification

Fixed size: 32 bytes. Format string: "<QQIB3xQ" (little-endian, no alignment padding)

Field	Type	Size	Byte offset
order_id	uint64	8	0 to 7
price	uint64	8	8 to 15
quantity	uint32	4	16 to 19
side	uint8	1	20
padding	-	3	21 to 23
timestamp_ns	uint64	8	24 to 31

Side: 0 = BUY, 1 = SELL

Cython Order Wrapper

The Cython wrapper is located at `ipc/order_wrapper.pyx`.

It accepts an existing order record in bytes format and checks that its length matches the 32-byte order record specification. It then uses the existing Python `unpack_order` function to decode the record.

The build configuration is provided in the root-level `setup.py` file.

Build status: The Cython order wrapper imports successfully in the current environment. The compiled extension build process has not been independently verified in this update.

Testing: The full Python test suite passed all 45 tests. Two dedicated order-wrapper tests passed, covering valid 32-byte record decoding and invalid-length handling.

Trade Record Specification

Fixed size: 64 bytes. Format string: "<QQIB3xQQQQ8x" (little-endian, no alignment padding).

Field	Type	Size	Byte offset
order_id	uint64	8	0 to 7
price	uint64	8	8 to 15
quantity	uint32	4	16 to 19
side	uint8	1	20
padding	-	3	21 to 23
timestamp_ns	uint64	8	24 to 31
trade_id	uint64	8	32 to 39
buyer_id	uint64	8	40 to 47

| seller_id | uint64 | 8 | 48 to 55 |

| padding | - | 8 | 56 to 63 |

Side: 0 = BUY, 1 = SELL

The trade record is implemented in `ipc/trade.py`.

`pack_trade()` encodes the fields into a fixed 64-byte record.

`unpack_trade()` decodes the record and raises `ValueError` if the input length is not 64 bytes.

report_engine.md

ChronosMatch Engine Report

1. Introduction

ChronosMatch is a zero-copy, high-frequency trading engine designed to process binary order records, match buy and sell orders, and publish executed trades with low overhead.

The engine uses Python for orchestration and a Cython-based matching core for optimized order-book operations.

2. System Architecture

The main processing flow is:

```
Order Generator
      |
      v
Order Ring Buffer (mmap)
      |
      v
Engine Loop
      |
      v
Matching Engine
      |
      v
Order Book Adapter
      |
      +------> Python Order Book
      |
      +------> Cython Book Core
      |
      v
Trade Ring Buffer
      |
      v
Statistics and Dashboard
```

The adapter selects the Cython implementation when the compiled extension is available. Otherwise, it falls back to the Python implementation.

3. Python Reference Order Book

The Python implementation is available in `engine/order_book.py`.

Its main responsibilities are:

• Maintaining separate bid and ask price levels.
• Matching incoming orders against existing opposite-side orders.
• Applying price-time priority.
• Supporting partial order fills.
• Keeping unfilled quantities in the order book.
• Providing the best bid, best ask and order-book snapshot.

Buy orders match with the lowest available sell price. Sell orders match with the highest available buy price.

Trades execute at the resting order's price. Orders at the same price are processed in FIFO order.

4. Cython Optimized Order Book

The optimized implementation is available in:

• `engine/book_core.pyx`
• `engine/book_adapter.py`

The Cython core uses C structures, pointers and dynamically allocated linked-list nodes to store orders.

Separate bid and ask arrays maintain price levels, with head and tail pointers supporting FIFO order insertion and removal.

Its main features include:

• C-level order storage.
• FIFO matching at each price level.
• Partial order execution.
• Best bid and best ask retrieval.
• Quantity calculation at individual price levels.
• Memory allocation and deallocation using `calloc()` and `free()`.

The default maximum supported price is 100000. Prices outside the configured range are rejected.

5. Matching Engine

The matching engine is implemented in `engine/matching_engine.py`.

Its responsibilities include:

1. Reading and unpacking incoming order records.
2. Calculating order latency when a valid timestamp is available.
3. Passing orders to the selected order book.
4. Generating trade IDs and timestamps.
5. Publishing executed trades to the trade ring buffer.
6. Updating order counts, trade counts and best bid/ask statistics.

If the trade ring buffer is full, the engine raises a `BufferError`.

6. Binary Data Format

ChronosMatch uses fixed-size binary records instead of JSON or Pickle for its order and trade messages.

Order Record

Defined in `ipc/record.py`.

Field	Size
Order ID	8 bytes
Price	8 bytes
Quantity	4 bytes
Side	1 byte
Padding	3 bytes
Timestamp	8 bytes
Total	32 bytes

Trade Record

Defined in `ipc/trade.py`.

Field	Size
Order ID	8 bytes
Price	8 bytes
Quantity	4 bytes
Side	1 byte
Padding	3 bytes
Timestamp	8 bytes
Trade ID	8 bytes
Buyer ID	8 bytes
Seller ID	8 bytes
Padding	8 bytes
Total	64 bytes

BUY is represented by 0 and SELL by 1.

7. Memory-Mapped Ring Buffer

The ring buffer is implemented in `ipc/ring_buffer.py` using Python's `mmap` module.

It provides:

• Fixed-size record storage.
• Head and tail position tracking.
• Push and pop operations.
• Full and empty buffer checks.
• Zero-copy record access through `peek_view()` and `advance()`.

The implementation follows a single-producer, single-consumer design.

The current engine loop uses the `pop()` method to consume order records.

8. Performance Benchmark

The benchmark is implemented in `engine/bench_engine.py`.

It measures order-book processing throughput, P99 latency and total elapsed time.

Test Configuration

• Random seed: 42
• Engine: CythonOrderBook
• Workloads: 1,000 and 1,000,000 orders

Benchmark Results

Metric	1,000 Orders	1,000,000 Orders
Throughput (orders/sec)	20027.80	11717.34
P99 Latency (ns)	220500	465100
Elapsed Time (sec)	0.0499306	85.3436218

Observations

• Both benchmark workloads completed successfully.
• Throughput decreased with the larger workload.
• P99 latency increased under the larger workload.
• The Cython order book was used in both runs.

These results measure the order-book benchmark path, not the complete ring-buffer processing pipeline.

9. Testing and Validation

The project includes tests covering:

• Python order-book matching.
• Cython order-book operations.
• Python and Cython matching parity.
• Order-book adapter selection.
• Engine-loop processing.
• Trade journal integration.
• Binary record handling.
• Ring-buffer operations.
• Statistics and dashboard functionality.

These tests help validate matching correctness, data handling and integration between project components.

10. Current Limitations

• The Cython order book uses a configured maximum price range.
• Finding the best available price involves scanning price levels.
• The current engine loop uses polling to check for incoming orders.
• The benchmark measures order-book performance rather than end-to-end pipeline latency.
• Performance can vary with workload size and system configuration.

11. Conclusion

ChronosMatch combines Python orchestration, Cython-based order matching, binary data records and memory-mapped ring buffers.

The project implements price-time priority, partial order execution, trade publication, performance measurement and monitoring support.

The Python reference implementation and optimized Cython implementation provide two matching paths, with automatic fallback when the compiled extension is unavailable.

The system provides a foundation for further optimization and integration testing of a low-latency trading engine.

benchmark_engine.md

ChronosMatch Engine Benchmark

Test Configuration

• Seed: 42
• Engine: CythonOrderBook
• Benchmark runs: 1,000 and 1,000,000 orders

Results

| Metric | 1,000 Orders | 1,000,000 Orders |

|---|---:|---:|

| Throughput (orders/sec) | 20027.80 | 11717.34 |

| P99 latency (ns) | 220500 | 465100 |

| Elapsed time (sec) | 0.0499306 | 85.3436218 |

Observations

• Both benchmark runs completed successfully.

• The larger workload had lower throughput.

• P99 latency increased under the larger workload.

• CythonOrderBook was used for both runs.

Methodology

Orders were generated using seed 42 and processed through the benchmark's order-book path.

These results measure the benchmarked order-book path, not the complete ring-buffer pipeline.

benchmark_results.md

ChronosMatch Benchmark Results

1. Simulator and Matching Engine Benchmark

This benchmark records the simulator and matching-engine output.

• Number of orders: 100

• Benchmark script: `benchmarks/benchmark_simulator.py`

| Metric | Result |

|---|---:|

| Orders generated | 100 |

| Orders processed | 100 |

| Trades produced | 77 |

| Orders in statistics | 100 |

| Best bid | 10026 |

| Best ask | 10034 |

| Last trade price | 10026 |

The simulator successfully generated and processed 100 orders. The matching engine produced 77 trades, and the statistics recorded all 100 orders.

2. Ring Buffer Access Benchmark

This benchmark compares copying access using `pop()` with zero-copy access using `peek_view()` followed by `advance()`.

Test configuration

Parameter	Value
Iterations	200,000
Ring buffer capacity	1,024 records
Record size	32 bytes
Benchmark script	`ipc/bench_ring.py`
Platform tested	Windows
Python version	3.14.7

Results

Operation	Execution Time	Throughput
Copying `pop()`	0.9053 seconds	220,909 operations/sec
Zero-copy `peek_view() + advance()`	1.0866 seconds	184,053 operations/sec

In this particular run, copying `pop()` achieved higher throughput than zero-copy access. Avoiding a data copy does not always guarantee higher measured performance because acquiring and releasing a memory view and advancing the buffer also have overhead.

Limitations

• The measurements represent individual Windows benchmark runs.
• Results may vary depending on system load and hardware.
• Linux benchmark results have not yet been verified.
• Repeated testing is required before drawing broader conclusions.

3. Overall Conclusion

The simulator benchmark demonstrates successful order processing and trade generation. The ring-buffer benchmark confirms that both copying and zero-copy access methods work for 200,000 iterations. These are separate measurements and should not be treated as a direct comparison of matching-engine latency.

setup.md

ChronosMatch Setup Guide

Prerequisites

• Python installed and available from the terminal

• Git

• Visual Studio Code (recommended)

1. Open the repository

Open a terminal at the root of the "chronosmatch" repository.

2. Create a virtual environment

On Windows PowerShell:

python -m venv .venv

3. Activate the environment

.\.venv\Scripts\Activate.ps1

If PowerShell blocks activation, use Command Prompt:

.venv\Scripts\activate.bat

4. Install dependencies

python -m pip install -r requirements.txt

5. Run the tests

python -m pytest -q

6. Dashboard

Refer to "dashboard/README.md" for dashboard instructions. On Windows, "windows-curses" is included in the project dependencies.

Troubleshooting

• If "python" is not recognised, check your Python installation and PATH.

• If a package installation fails, check your internet connection and Python version.

• Run commands from the repository root unless otherwise specified.

A complete live demonstration requires the simulator, matching engine, shared statistics, and dashboard to be integrated and running.

usage_guide.md

ChronosMatch Usage Guide

Overview

ChronosMatch is a zero-copy high-frequency trading engine designed to process buy and sell orders using a shared-memory ring buffer.

This guide explains how to set up, test, and run the project.

1. Open the Project

Open the ChronosMatch repository in Visual Studio Code.

Open the terminal and navigate to the project root directory.

2. Set Up the Environment

Create a Python virtual environment:

python -m venv .venv

Activate the environment:

.\.venv\Scripts\Activate.ps1

Install the required dependencies:

python -m pip install -r requirements.txt

3. Run the Tests

Run the following command to check whether the project is working correctly:

python -m pytest -v

The tests check different components, including the ring buffer and simulator.

4. Run the Benchmark

Run the benchmark script:

python benchmarks/benchmark_simulator.py --count 100

The benchmark generates and processes 100 orders and displays the resulting trade and market statistics.

5. Dashboard

Refer to "dashboard/README.md" for instructions on running the market dashboard.

The dashboard displays market information such as order book details, activity, and engine status.

6. Troubleshooting

• If Python is not recognized, check that Python is installed and added to PATH.
• If dependencies fail to install, check your internet connection and Python version.
• If tests fail, review the error messages in the terminal.
• If the dashboard does not start, refer to "dashboard/README.md".

Important Note

A complete live demonstration requires the simulator, matching engine, shared statistics, and dashboard to be integrated and running.

Refer to "docs/setup.md" for additional setup instructions.